

Chuyên đề: Cây Phân Đoạn & Kỹ Thuật Đánh Dấu Trễ (Segment Tree & Lazy Propagation)

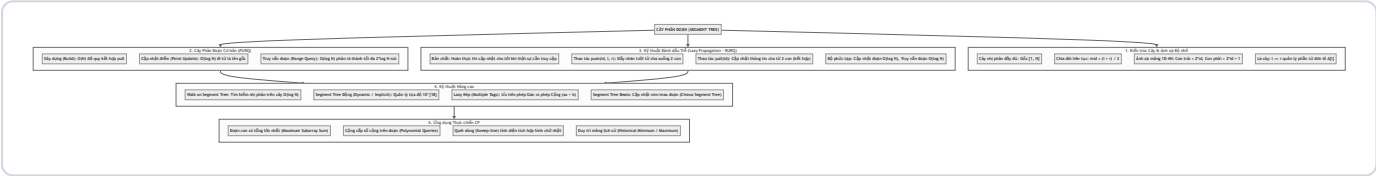
(Binary Segment Tree, Point/Range Updates, Lazy Propagation, Walk on Tree & Advanced Tagging)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Kiến Trúc & Bản Chất Toán Học của Cây Phân Đoạn

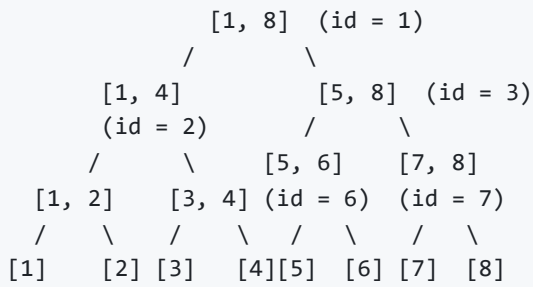
1.1. Khái niệm & Cấu trúc Phân nhánh

Cây Phân Đoạn (Segment Tree) là một cấu trúc dữ liệu cây nhị phân hoàn chỉnh được thiết kế để duy trì thông tin trên các đoạn con của một mảng và hỗ trợ:

- **Cập nhật dữ liệu** (điểm hoặc đoạn) trong $O(\log N)$.
- **Trả lời truy vấn** tổng hợp trên đoạn $[u, v]$ bất kỳ trong $O(\log N)$.

Mỗi nút trên cây quản lý một đoạn chỉ số $[l, r]$:

- **Nút gốc (Root):** Quản lý toàn bộ mảng $[1, N]$.
- **Nút lá (Leaf):** Quản lý đoạn suy biến gồm đúng 1 phần tử $[i, i]$.
- **Nút trong (Internal node):** Quản lý đoạn $[l, r]$ với $l < r$. Nút này luôn có đúng 2 nút con:
 - Nút con trái: Quản lý nửa đoạn bên trái $[l, mid]$ với $mid = \lfloor \frac{l+r}{2} \rfloor$.
 - Nút con phải: Quản lý nửa đoạn bên phải $[mid + 1, r]$.



1.2. Ánh Xạ Bộ Nhớ Trên Mảng 1D & Chứng Minh Kích Thước $4N$

Thay vì dùng con trỏ (`Node* left, right`), Segment Tree tĩnh thường được ánh xạ trực tiếp vào một mảng 1 chiều 1-indexed:

- Nút cha: `id`.
- Nút con trái: `2 * id` (hoặc `id << 1`).
- Nút con phải: `2 * id + 1` (hoặc `(id << 1) | 1`).
- Nút cha của `id`: `id / 2` (hoặc `id >> 1`).

📌 Quan trọng:

****Tại sao kích thước mảng Segment Tree phải là $4N$?****

1. Nếu N là lũy thừa của 2 ($N = 2^k$), Segment Tree là một cây nhị phân đầy đủ có độ cao $k = \log_2 N$. Tổng số nút là:

$$\sum_{i=0}^k 2^i = 2^{k+1} - 1 = 2N - 1 < 2N$$

2. Nếu N ****không phải là lũy thừa của 2****, ta cần mở rộng cây đến lũy thừa của 2 nhỏ nhất lớn hơn N , đặt là $2^{\lceil \log_2 N \rceil}$. Trong trường hợp xấu nhất ($N = 2^k + 1$), lũy thừa 2 tiếp theo là $2^{k+1} \approx 2N$. Số nút của cây khi đó là:

$$2 \cdot 2^{k+1} - 1 = 4 \cdot 2^k - 1 < 4N$$

****Kết luận:**** Khai báo mảng cây với kích thước $4N$ phần tử luôn ****đảm bảo an toàn tuyệt đối 100%****, không bao giờ bị tràn chỉ số (Out-of-bounds)!

2. Cây Phân Đoạn Cơ Bản: Point Update & Range Query (PURQ)

2.1. Thao tác Xây dựng Cây (Build) — $O(N)$

Thuật toán đệ quy đi từ gốc xuống các lá, khởi tạo giá trị tại lá rồi kết hợp thông tin (hàm `pull` hay `combine`) ngược lên gốc:

```

void build(int id, int l, int r) {
    if (l == r) {
        tree[id] = a[l];
        return;
    }
    int mid = (l + r) / 2;
    build(2 * id, l, mid);
    build(2 * id + 1, mid + 1, r);
    tree[id] = combine(tree[2 * id], tree[2 * id + 1]); // pull
}

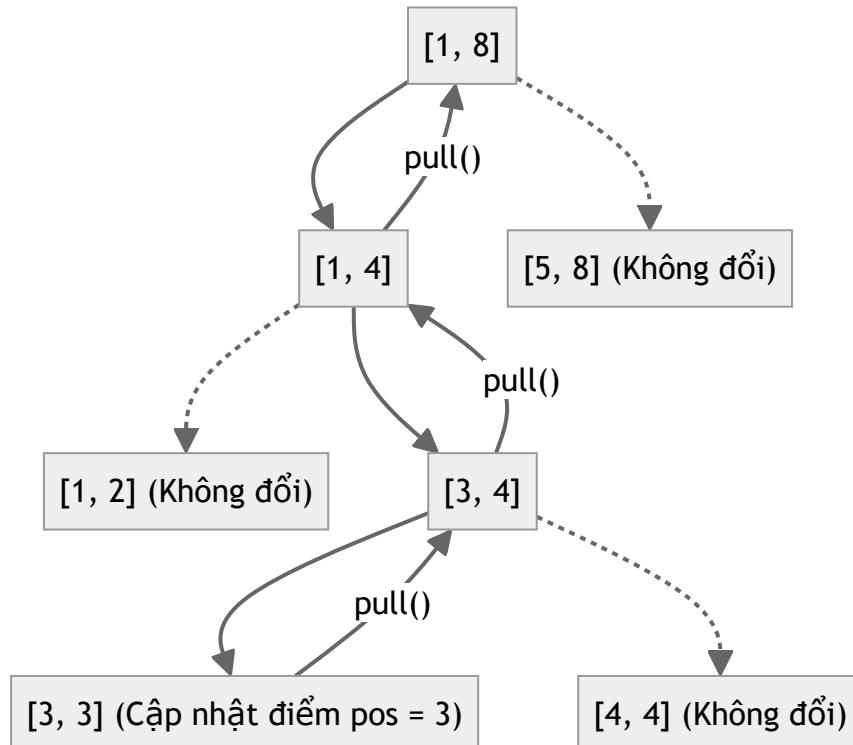
```

- **Chứng minh độ phức tạp:** Tổng số nút được thăm bằng đúng số nút trên cây $\leq 4N \implies O(N)$ thời gian.

2.2. Thao tác Cập nhật Điểm (Point Update) — $O(\log N)$

Khi phần tử tại vị trí pos thay đổi giá trị ($A[pos] = val$ hoặc $A[pos] += val$):

1. Từ gốc, so sánh pos với mid :
 - Nếu $pos \leq mid$: Đệ quy sang cây con trái.
 - Nếu $pos > mid$: Đệ quy sang cây con phải.
2. Khi chạm tới nút lá ($l = r = pos$), cập nhật giá trị.
3. Trên đường quay lui đệ quy (Backtracking), cập nhật lại giá trị của các nút tổ tiên bằng `combine`.



Độ sâu của cây là $\lceil \log_2 N \rceil \implies$ chỉ có đúng $\log_2 N + 1$ nút bị thay đổi $\implies$ Thời gian: $O(\log N)$.

2.3. Thao tác Truy vấn Đoạn (Range Query) — $O(\log N)$

Khi cần tính giá trị tổng hợp trên đoạn $[u, v]$:

Xét nút hiện tại tại quản lý đoạn $[l, r]$:

- **Trường hợp 1 (Rời nhau hoàn toàn):** $r < u$ hoặc $l > v \implies$ Trả về phần tử trung hòa (Identity element: 0 cho tổng, $+\infty$ cho min, $-\infty$ cho max).

- **Trường hợp 2 (Nằm trọn vẹn bên trong):** $u \leq l$ và $r \leq v \implies$ Đoạn $[l, r]$ nằm hoàn toàn trong $[u, v]$, trả về ngay giá trị `tree[id]`.

- **Trường hợp 3 (Giao nhau một phần):** Chia đôi đoạn, đệ quy sang cả 2 con và kết hợp kết quả:

$$\text{query}(u, v) = \text{combine}(\text{query con trái}, \text{query con phải})$$

i Ghi chú:

****Định lý Số Lượng Nút Được Thăm:****

Tại mỗi tầng của cây, truy vấn $[u, v]$ chỉ thăm ****tối đa 4 nút****.

Do cây có độ cao $\approx \log_2 N$, tổng số nút được thăm không bao giờ vượt quá $4 \log_2 N \implies$ Thời gian truy vấn được đảm bảo chặt chẽ là **** $O(\log N)$ ****.

3. Kỹ Thuật Đánh Dấu Trễ (Lazy Propagation) — RURQ $O(\log N)$

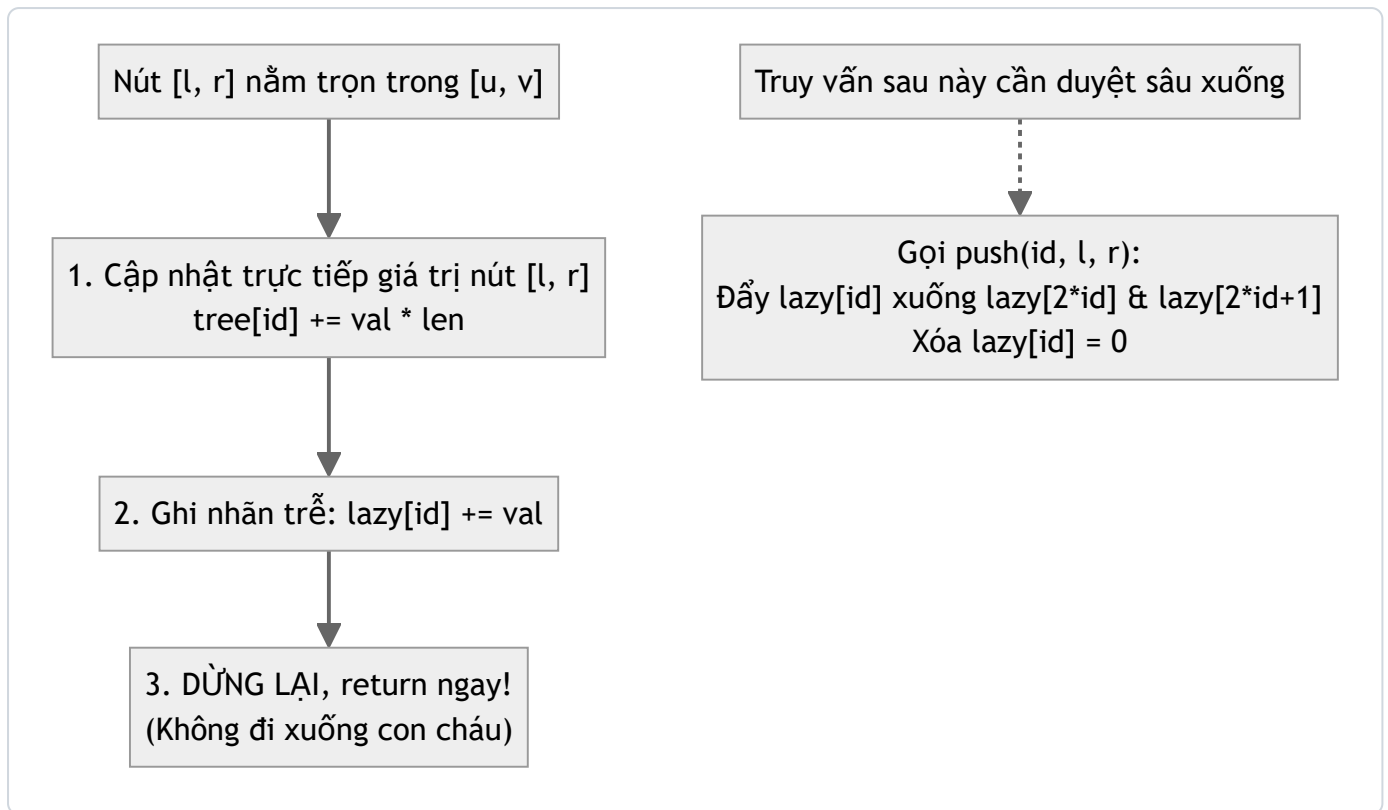
Khi bài toán yêu cầu **cập nhật trên cả một đoạn** $[u, v]$ (Range Update) và **truy vấn trên đoạn** (Range Query), nếu dùng Point Update thì mỗi lần cập nhật sẽ tốn $O(N \log N)$ — hoàn toàn không thể chấp nhận được trong các bài toán thi đấu.

3.1. Bản Chất của Lazy Propagation

Triết lý: "Đừng làm việc gì cho tới khi thực sự bị ép phải làm."

Khi một đoạn $[l, r]$ của cây nằm trọn vẹn trong khoảng cập nhật $[u, v]$:

1. Ta cập nhật giá trị của chính nút đó ngay lập tức.
2. Ta **không** đi sâu xuống các nút con cháu của nó.
3. Thay vào đó, ta lưu lại một **nhãn lười (Lazy Tag)** tại nút đó để ghi nhớ: "Các nút con của ta cần phải được cập nhật lượng này trong tương lai".
4. Chỉ khi có một truy vấn hoặc cập nhật sau này cần đi sâu vào các con của nút này, ta mới **đẩy nhãn lười xuống (Push down)** một tầng cho 2 nút con trực tiếp.



3.2. Quy Trình Hai Thao Tác Thần Thánh: `push` và `pull`

1. Thao tác `push(id, l, r)` — Truyền nhãn từ cha xuống con

Được gọi ở **đầu mỗi hàm** `update` và `query` trước khi chuẩn bị rẽ nhánh xuống cây con:

```
void push(int id, int l, int r) {
    if (lazy[id] == 0) return; // Không có nhãn lười
    int mid = (l + r) / 2;

    // Đẩy xuống con trái
    tree[2 * id] += lazy[id] * (mid - l + 1);
    lazy[2 * id] += lazy[id];

    // Đẩy xuống con phải
    tree[2 * id + 1] += lazy[id] * (r - mid);
    lazy[2 * id + 1] += lazy[id];

    // Xóa nhãn tại cha
    lazy[id] = 0;
}
```

2. Thao tác `pull(id)` — Kéo thông tin từ con lên cha

Được gọi ở **cuối hàm** `update` sau khi cả 2 nhánh con đã hoàn thành cập nhật:

```
void pull(int id) {
    tree[id] = tree[2 * id] + tree[2 * id + 1]; // hoặc min, max
}
```

3.3. Bảng Trace Step-by-Step Cập Nhật & Truy Vấn Đoạn với Lazy

Xét mảng $N = 4$ phần tử, ban đầu toàn số 0: $A = [0, 0, 0, 0]$.

Thực hiện 2 thao tác:

1. `Update([1, 3], +5)`
2. `Query([2, 4])`

Thao tác	Nút được duyệt	Đoạn $[l, r]$	Tình trạng	<code>tree[id]</code>	<code>lazy[id]</code>	Giải thích hành động
Update([1, 3], +5)	$id = 1$	$[1, 4]$	Giao một phần	—	0	$mid = 2$, rẽ trái sang $[1, 2]$ và phải sang $[3, 4]$
	$id = 2$	$[1, 2]$	Nằm trọn trong $[1, 3]$	$0 \rightarrow 10$	$0 \rightarrow 5$	Cập nhật $tree = 0 + 5 \times 2 = 10$, gán $lazy = 5$, return
	$id = 3$	$[3, 4]$	Giao một phần	—	0	$mid = 3$, rẽ trái sang $[3, 3]$ và phải sang $[4, 4]$
	$id = 6$	$[3, 3]$	Nằm trọn trong $[1, 3]$	$0 \rightarrow 5$	$0 \rightarrow 5$	Cập nhật $tree = 0 + 5 \times 1 = 5$, gán $lazy = 5$, return
	$id = 7$	$[4, 4]$	Ngoài khoảng	0	0	Không làm gì, return
	$id = 3$	$[3, 4]$	Backtrack	$5 + 0 = 5$	0	<code>pull(3)</code> : $tree[3] = tree[6] + tree[7] = 5$
	$id = 1$	$[1, 4]$	Backtrack	$10 + 5 = 15$	0	<code>pull(1)</code> : $tree[1] = tree[2] + tree[3] = 15$
Query([2, 4])	$id = 1$	$[1, 4]$	Giao một phần	15	0	$mid = 2$, rẽ sang $[1, 2]$ và $[3, 4]$
	$id = 2$	$[1, 2]$	Giao một phần	10	5	Có nhãn lazy=5! Gọi push(2) $\Rightarrow$ đẩy nhãn cho $[1, 1]$ và $[2, 2]$
	$id = 5$	$[2, 2]$	Nằm trong $[2, 4]$	5	5	Lấy giá trị 5 đóng góp vào tổng
	$id = 3$	$[3, 4]$	Nằm trọn trong $[2, 4]$	5	0	Nằm trọn vẹn, lấy luôn giá trị $tree[3] = 5$

Thao tác	Nút được duyệt	Đoạn $[l, r]$	Tình trạng	<code>tree[id]</code>	<code>lazy[id]</code>	Giải thích hành động
	Kết quả	—	—	$5 + 5 = 10$	—	Kết quả chính xác: $A = [5, 5, 5, 0] \implies \sum_2^4 = 10$

4. Lazy Kép: Kết Hợp Cả Phép Gán Đè Và Phép Cộng Dồn ($a \cdot x + b$)

Trong các bài toán mức độ ICPC National / Regional (như CSES 1735 - Range Updates and Sums), một đoạn có thể vừa nhận thao tác **cộng dồn** ($A[i] += v$), vừa nhận thao tác **gán đè** ($A[i] = v$).

Mô hình Đại số Ánh Xạ Tuyến Tính $f(x) = a \cdot x + b$

Thay vì duy trì 2 cờ độc lập dễ bị xung đột logic, ta mô hình hóa mọi thao tác dưới dạng phép biến đổi affine:

- Ban đầu (Không có nhãn): $a = 1, b = 0 \implies f(x) = x$.
- Thao tác cộng $+v$: $a' = 1, b' = v \implies f(x) = 1 \cdot x + v$.
- Thao tác gán $= v$: $a' = 0, b' = v \implies f(x) = 0 \cdot x + v$.

Quy tắc hợp thành hai nhãn liên tiếp:

Nếu một nút đang có nhãn $f(x) = a_1 \cdot x + b_1$ và nhận thêm một nhãn mới $g(x) = a_2 \cdot x + b_2$:

$$g(f(x)) = a_2 \cdot (a_1 \cdot x + b_1) + b_2 = (a_2 \cdot a_1) \cdot x + (a_2 \cdot b_1 + b_2)$$

Nhãn mới thu được có:

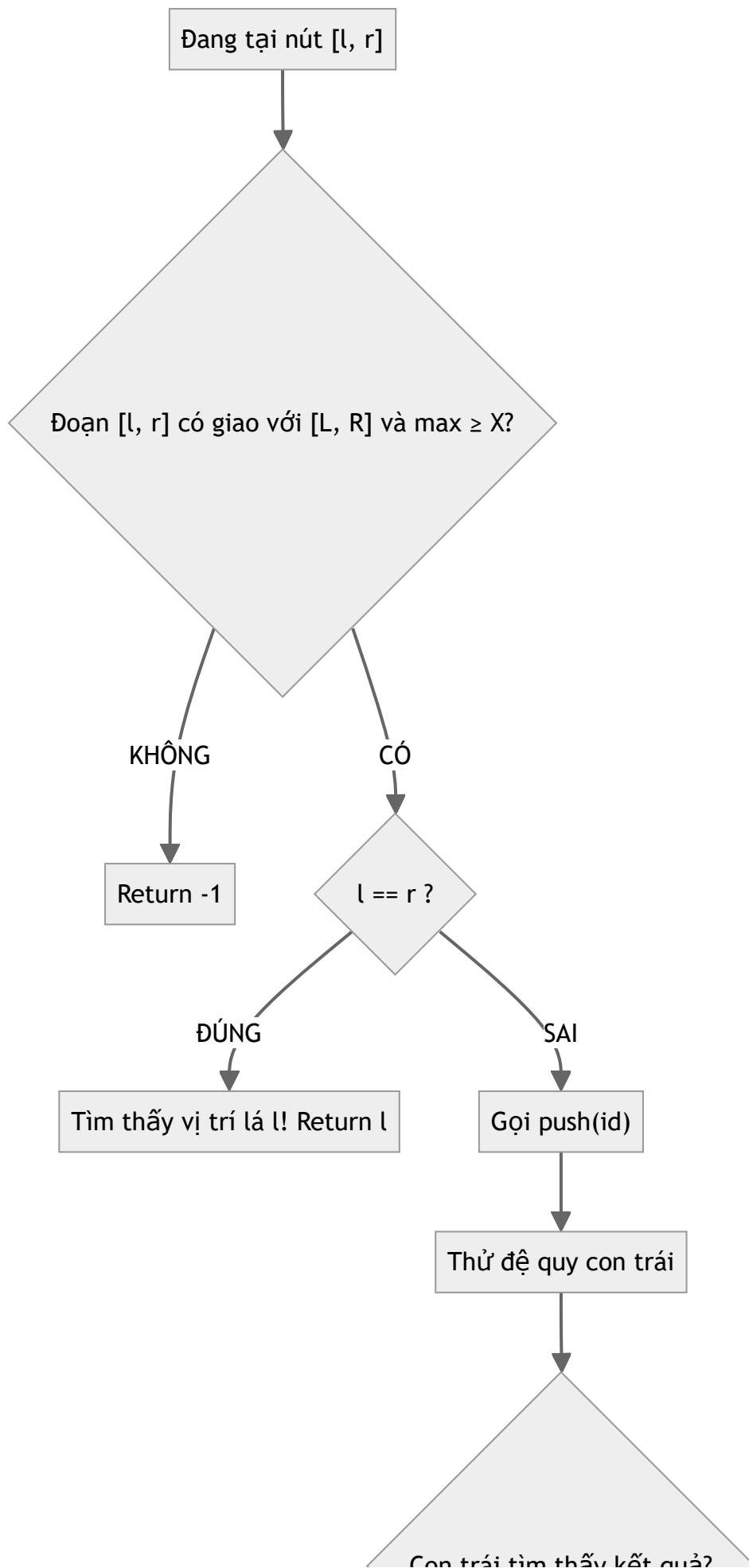
$$\begin{cases} a_{\text{new}} = a_2 \cdot a_1 \\ b_{\text{new}} = a_2 \cdot b_1 + b_2 \end{cases}$$

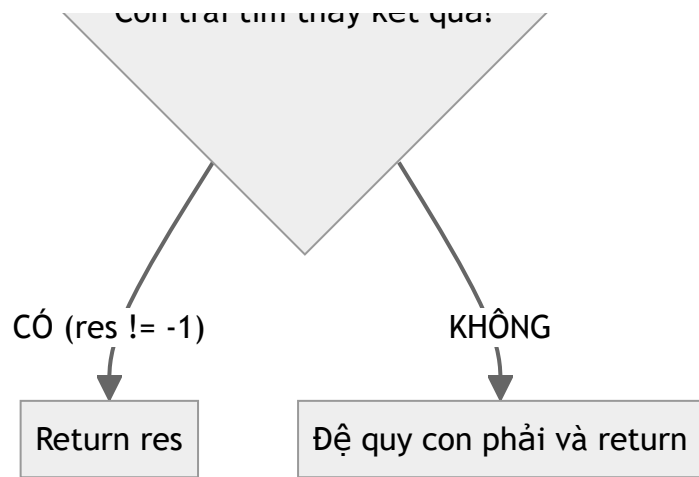
Mô hình hóa toán học này giúp loại bỏ hoàn toàn các cấu trúc `if/else` lằng nhằng và tránh 100% các bug logic khi kết hợp nhiều loại nhãn lười!

5. Walk on Segment Tree: Tìm Kiếm Nhị Phân $O(\log N)$ Trên Cây

Nhiều bài toán yêu cầu: "Tìm vị trí đầu tiên $\geq L$ có giá trị $\geq X$ " hoặc "Tìm tiền tố nhỏ nhất có tổng $\geq S$ ".

- **Cách tiếp cận ngây thơ:** Viết hàm `binary_search` bên ngoài, mỗi bước gọi `query(L, mid)` của Segment Tree $\implies$ Độ phức tạp $O(\log^2 N)$.
- **Kỹ thuật Walk on Segment Tree:** Tận dụng trực tiếp cấu trúc cây nhị phân để đi từ gốc xuống lá, đạt độ phức tạp tối ưu tuyệt đối $O(\log N)$!





```

int find_first(int id, int l, int r, int ql, int qr, auto&& pred) {
    if (l > qr || r < ql || !pred(tree[id])) return -1;
    if (l == r) return l;
    push(id, l, r);
    int mid = (l + r) / 2;
    int res = find_first(2 * id, l, mid, ql, qr, pred);
    if (res != -1) return res;
    return find_first(2 * id + 1, mid + 1, r, ql, qr, pred);
}

```

6. Segment Tree Động (Dynamic / Implicit Segment Tree)

Khi khoảng tọa độ của bài toán lên tới $N = 10^9$ hoặc 10^{18} (không thể tạo mảng $4N$):

- Nếu các truy vấn không biết trước (Online) hoặc không thể nén tọa độ: Ta chỉ tạo các nút của cây **khi nút đó thực sự được ghé thăm**.
- Mỗi nút chứa 2 con trỏ hoặc 2 chỉ số trỏ tới nút con: `ls` (left child) và `rs` (right child).
- Với Q thao tác, số lượng nút được tạo thêm tối đa là $Q \cdot \log(\text{MAX_VAL}) \approx 10^5 \cdot 30 \approx 3 \times 10^6$ nút, tiêu tốn chỉ khoảng 50 MB RAM!

```

struct Node {
    int ls = 0, rs = 0;
    long long sum = 0, lazy = 0;
};
std::vector<Node> tree; // Cấp phát động theo mảng (Node Pool)

```

7. Cạm Bẫy Phòng Thi & Kỹ Thuật Tối Ưu (Traps & Best Practices)

1. Quên gọi `push` trước khi truy cập con:

Bất cứ khi nào bạn chuẩn bị gọi `tree[2 * id]` hoặc chia đôi `mid` để đệ quy xuống dưới, **bắt buộc phải gọi `push(id, l, r)` ngay phía trên!** Nếu quên, con của bạn sẽ nhận dữ liệu lỗi thời.

2. Tràn số `long long` khi nhân độ dài đoạn:

Trong phép cộng đoạn: `tree[id] += lazy[id] * (r - l + 1)`. Nếu `lazy[id] = 10^9` và $(r - l + 1) =$

10^5 , tích này lên tới 10^{14} , vượt quá kiểu `int` 32-bit. Toàn bộ các trường `sum` và `lazy` phải khai báo `long long`.

3. Giá trị trả về của trường hợp ranh giới ngoài:

Khi $r < u$ hoặc $l > v$, hàm trả về phần tử trung hòa. Nếu bài toán tìm `min`, trả về `INF = 2e18`; nếu tìm `max`, trả về `-INF`; nếu tìm tổng, trả về `0`. Đặt nhầm giá trị trung hòa là nguyên nhân số 1 gây WA.

4. Tránh khởi tạo lại cây $O(N)$ trong nhiều test case:

Khi bài toán có T test case và $\sum N \leq 2 \times 10^5$, không dùng `memset` toàn bộ mảng $4 \cdot \text{MAX_N}$. Chỉ xóa đúng $4 \cdot N$ phần tử của test case hiện tại để tránh bị TLE.

8. Tài liệu Tham khảo & Đọc thêm

1. CP-Algorithms:

2. [Segment Tree](#) — Cập nhật điểm, truy vấn đoạn và cây phân đoạn động.

3. VNOI Wiki:

4. [Cây phân đoạn cơ bản](#) — Cấu trúc dữ liệu $4N$ và thao tác PURQ.

5. [Segment Tree mở rộng & Lazy Propagation](#) — Kỹ thuật đánh dấu trễ push/pull.

6. USACO Guide (Platinum Section):

7. [Advanced Segment Trees](#) — Walk on Segment Tree và lazy kết hợp đa phép toán.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.